Flight Delay Prediction with Entity Embeddings and a Multilayer Perceptron: Phase 2 — Deep Learning

Asad Ali, Divyansh Bhartia, Vikrant Yadav, Mayank Pillai, Prashant Raj
CSAI Project 21
Deep Learning

Abstract—We extend our Phase 1 flight delay study with a deep learning model. The task is unchanged: predict, before departure, whether a flight from a New York City airport in 2013 will arrive 15 or more minutes late. We train a compact multilayer perceptron (MLP) with learned entity embeddings for the carrier, origin and destination, on the same leakage-free features and date-based split as Phase 1. The network has 15,361 parameters and is regularised with dropout, decoupled weight decay and early stopping. On the November–December test set it reaches a PR AUC of 0.386 (random baseline 0.258) and ROC AUC of 0.653, between Logistic Regression (0.376) and XGBoost (0.391), and is stable across five random seeds (0.385 ± 0.003). Its raw probabilities are the best calibrated of all models (Brier score 0.182, against 0.188 for calibrated XGBoost). An ablation study over five seeds shows that the recent delay-history features matter most (PR AUC falls from 0.3845 to 0.3575 without them), while embeddings perform on par with one-hot encoding. Finally, an offline what-if analysis finds a lower-risk departure slot for 92.7% of the highest-risk flights, mostly two hours earlier.

Index Terms—flight delay prediction, deep learning, entity embeddings, multilayer perceptron, dropout, calibration, ablation

I. Introduction

Flight delays cost airlines and passengers time and money and spread through the network as late aircraft make later flights late. In Phase 1 we built a leakage-free pipeline that predicts, before departure, whether a flight will arrive at least 15 minutes late, and compared Logistic Regression with XGBoost. XGBoost was best, with a test PR AUC of 0.391.

Phase 2 asks whether a deep learning model can learn the same patterns, and what its parts contribute. The Project 21 brief suggests a compact MLP with learned embeddings for airline, origin and destination; a temporal model is optional and was not used. Our contributions are:

- an MLP with entity embeddings trained on exactly the Phase 1 features, split and metrics, so that the comparison with the Phase 1 models is fair;
- a regularisation setup (dropout, AdamW weight decay, early stopping) and a stability check over five seeds;
- an ablation study of the embeddings, categorical inputs, history features and regularisation;
- a calibration analysis, a look at what the embeddings learned, and an offline what-if rescheduling analysis with explicit assumptions.

II. Deep Learning Literature Review

Deep learning for flight delays. Sternberg et al. [1] review flight delay prediction and note a shift from statistical models towards machine learning. Kim et al. [2] apply recurrent neural networks to day-to-day airport delay sequences, showing that deep models can capture delay dynamics when ordered history is available.

Multilayer perceptrons. An MLP stacks linear layers and non-linear activations and is trained by back-propagation of the loss gradient [3]. Rectified linear units (ReLU) [4] avoid saturating gradients and are the default hidden activation.

Entity embeddings. Learned dense vectors for discrete symbols were popularised for words by Mikolov et al. [5]. Guo and Berkhahn [6] apply the same idea to categorical variables in tabular data: each category is mapped to a trainable vector, so that categories with similar effects can end up close together and share statistical strength. This is the basis of our carrier, origin and destination inputs.

Regularisation and optimisation. Dropout [7] randomly removes hidden units during training to reduce co-adaptation and overfitting. Adam [8] adapts the step size per parameter; AdamW [9] decouples weight decay from the gradient update, which makes L2-style regularisation behave as intended. Early stopping [10] keeps the model from the epoch with the best validation score.

Deep learning versus trees on tabular data. Several studies find that gradient-boosted trees such as XGBoost [11] still match or beat deep networks on medium-sized tabular data [12], [13]. Reasons include irregular target functions, uninformative features and the rotation invariance of MLPs. We therefore treat XGBoost as a strong reference, not a weak baseline.

Calibration. Guo et al. [14] show that modern neural networks are often over-confident and propose temperature scaling, which divides the logits by one learned constant. Isotonic regression [15] is the non-parametric alternative we used in Phase 1.

III. Dataset and Preprocessing

A. Data, features and split

We use the Flights 2013 dataset (nycflights13 [16]) with the Phase 1 pipeline unchanged: 327,346 completed flights, 24.5% delayed; all post-departure fields removed; features for calendar and schedule (weekend flag, distance, and cyclic sin / cos encodings of the departure hour, arrival hour and day of week), airport congestion (flights scheduled from the origin in the same hour and on the same day, counted on the full schedule including later-cancelled flights) and recent history (the share of delayed flights over the previous seven days for the carrier, origin, destination and route, today excluded). The data are split by date: train January–August (217,727 flights, 26.4% delayed), validation September–October (55,628, 15.7%) and test November–December (53,991, 25.8%).

B. Preprocessing for the network

Categorical inputs. Carrier, origin and destination are mapped to integer ids built from the training set only. Id 0 is reserved for categories never seen in training; 57 test flights go to such a destination and use this shared "unknown" vector.

Numeric inputs. The 14 numeric features are standardised to zero mean and unit variance with training statistics; missing history values (days without history) are filled with the training mean first.

Class imbalance. Unlike Phase 1, the network is trained with the plain binary cross-entropy, without class weights. Weighting changes the predicted probabilities but not their ranking, so PR and ROC AUC are barely affected, while unweighted training keeps the probabilities close to the true delay rate.

IV. Architecture

For a flight with numeric vector $x \in \mathbb{R}^{14}$ and category ids c_1, c_2, c_3, each id selects a row of an embedding matrix E_j, giving vectors of size 8 (carrier, 17 ids), 2 (origin, 4 ids) and 16 (destination, 103 ids). The sizes are small and grow with the number of categories. The input is the concatenation

$$h_0 = [\,x;\; E_1[c_1];\; E_2[c_2];\; E_3[c_3]\,] \in \mathbb{R}^{40}. \qquad (1)$$

Two hidden layers with ReLU and dropout ($p = 0.3$) follow,

$$h_k = \mathrm{Dropout}\big(\mathrm{ReLU}(W_k h_{k-1} + b_k)\big), \quad k = 1, 2, \qquad (2)$$

with 128 and 64 units, and a linear output gives the logit $z = w^\top h_2 + b$ and the probability $\hat{p} = \sigma(z)$. The model has 15,361 parameters, of which 1,792 are embedding weights. It is trained to minimise the binary cross-entropy

$$\mathcal{L} = -\frac{1}{N} \sum_i \big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)\big]. \qquad (3)$$

The embeddings are learned jointly with the rest of the network by back-propagation, so they encode whatever differences between categories help the prediction.

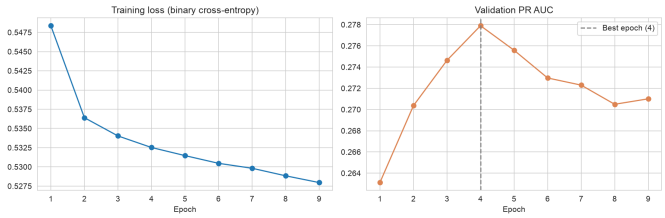


Fig. 1. Training loss and validation PR AUC per epoch (seed 0). The best epoch (4) is kept.

TABLE I
Test set results (November–December 2013)

Model	PR AUC	ROC AUC	Rec@P0.5	Brier
Random guess	0.258	0.500	–	–
Log. Regression	0.376	0.645	0.059	0.228
XGBoost	0.391	0.656	0.111	0.219
MLP (embeddings)	0.386	0.653	0.094	0.182

V. Training and Regularisation

We use AdamW [9] with learning rate 10^{-3} and weight decay 10^{-4}, mini-batches of 1,024 flights, and at most 30 epochs. After each epoch we compute the validation PR AUC and keep the best weights; training stops after five epochs without improvement [10]. Models are implemented in PyTorch [17] and trained on a laptop CPU in a few seconds per model.

Fig. 1 shows a typical run. The training loss keeps falling, but the validation PR AUC peaks at epoch 4 and then declines: after a few epochs the network starts to fit patterns specific to the training months. Early stopping, dropout and weight decay limit this overfitting.

VI. Technical Validation

A. Stability across seeds

Weight initialisation, batch order and dropout are random. Over five seeds the test PR AUC is 0.3845±0.0027 (range 0.380–0.387) and the ROC AUC 0.649–0.653, so differences smaller than about 0.005 between configurations should not be read as real.

B. Comparison with Phase 1

Table I and Fig. 2 compare the MLP (seed 0) with the Phase 1 models on the test set. The MLP clearly beats Logistic Regression and is slightly behind XGBoost in PR AUC, ROC AUC and recall at 50% precision (9.4% vs 11.1% of delayed flights). This matches the literature on tabular data [12], [13]: with 14 engineered features and three categorical inputs, boosted trees remain very strong.

C. Calibration

Without class weights, the MLP's raw probabilities are already well calibrated (Fig. 3): its Brier score of 0.182 is the best of all models, and its mean prediction (0.244) is close to the test delay rate (0.258). We also fitted

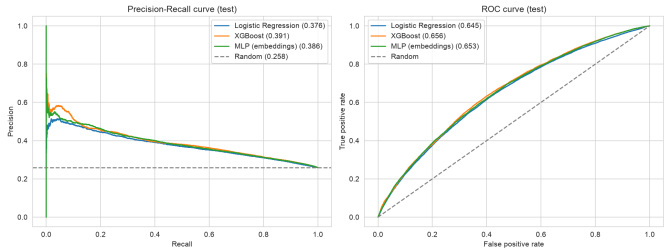


Fig. 2. Precision–recall and ROC curves on the test set.

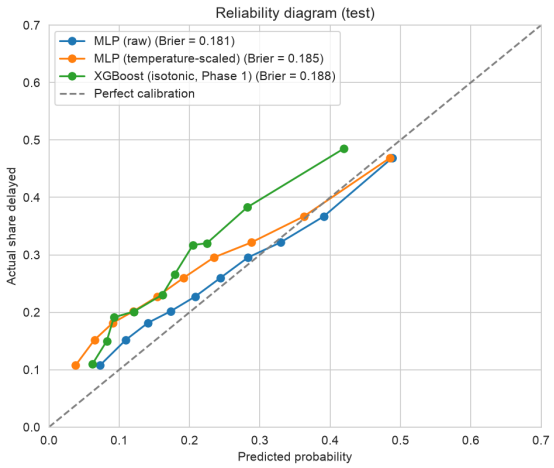


Fig. 3. Reliability diagram on the test set.

temperature scaling [14] on the validation set. The learned temperature $T = 0.785$ sharpened the probabilities and lowered their mean to 0.203, raising the Brier score to 0.185. The cause is the seasonal shift: September–October had far fewer delays (15.7%) than November–December (25.8%), so any calibration fitted on them pulls predictions down. The isotonic-calibrated XGBoost from Phase 1 suffers from the same effect (Brier 0.188, mean prediction 0.175). We therefore keep the MLP's raw probabilities.

VII. Ablation Study

Each variant was trained with five seeds; Table II and Fig. 4 report the mean test PR AUC with its standard deviation.

- History features are by far the most important input: removing them lowers the test PR AUC from 0.3845 to 0.3575 and the ROC AUC from 0.6517 to 0.6248.
- Embeddings perform on par with one-hot encoding and with no categorical inputs at all; all three are within one standard deviation. The recent delay rates of the carrier, origin, destination and route already summarise most of what the categories add.
- Regularisation gives a small gain and, more clearly, more stable results: without dropout and weight decay the seed-to-seed standard deviation roughly doubles (0.0061 vs 0.0027).

TABLE II
Ablation study (5 seeds, test set)

Variant	Val PR AUC	Test PR AUC	Test ROC
Full model (emb.)	0.2730	0.3845 ± 0.0027	0.6517
One-hot, no emb.	0.2691	0.3862 ± 0.0037	0.6565
No categorical	0.2725	0.3857 ± 0.0041	0.6553
No history	0.2679	0.3575 ± 0.0017	0.6248
No dropout / w. decay	0.2687	0.3826 ± 0.0061	0.6499

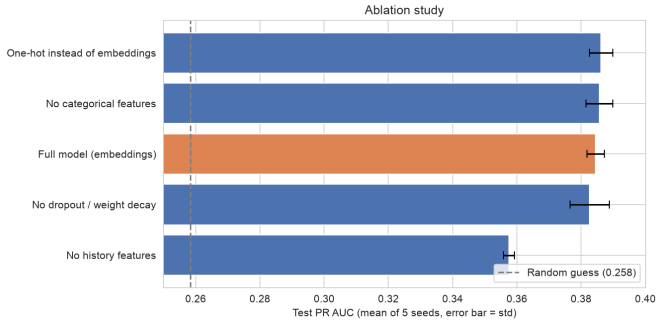


Fig. 4. Ablation study: mean test PR AUC over five seeds (error bars: one standard deviation).

The full model has the best validation PR AUC, which is how it was chosen; the test differences between the first three variants are within noise.

VIII. What Did the Embeddings Learn?

Fig. 5 projects the eight-dimensional carrier embeddings onto their first two principal components. The first component correlates only weakly with each carrier's training delay rate ($r = -0.25$). This agrees with the ablation: because the carrier's recent delay rate is already an input, the embeddings do not need to encode a carrier's overall delay level and instead capture smaller differences, for example in how a carrier's delays depend on the time of day. Carrier OO, with only seven training flights, is an outlier with an unreliable delay rate.

IX. What-If Rescheduling Analysis

The Project 21 brief asks for an offline rescheduling analysis based on explicit assumptions. We take the 10% of test flights with the highest predicted risk (5,400 flights, risk ≥ 0.429) and re-score each one with its departure moved by -2, -1, $+1$ or $+2$ hours, assuming that: the flight stays on the same day, departs between 05:00 and 23:59 and does not arrive after midnight; the arrival moves by the same amount; route, carrier, date and history features are unchanged; congestion is recomputed for the new hour; flights are moved one at a time; and crews, aircraft rotations, gates and passengers are ignored. A move is recommended only if it lowers the predicted risk.

Under these assumptions, 92.7% of high-risk flights have a lower-risk slot, and for 40.7% the risk falls by at least five percentage points. The mean predicted risk of the group

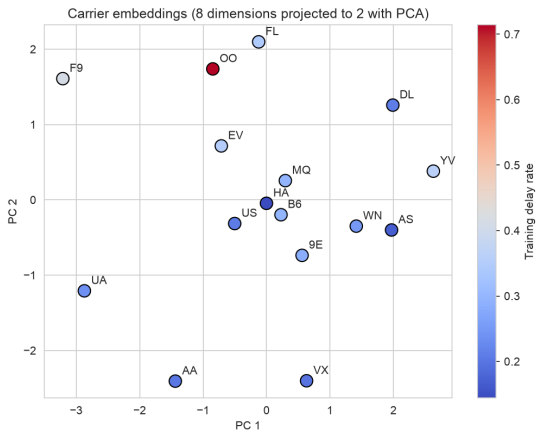


Fig. 5. Carrier embeddings projected to two dimensions with PCA, coloured by training delay rate.

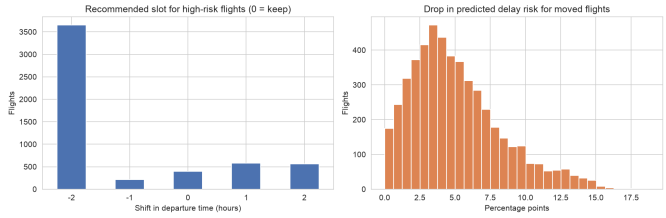


Fig. 6. What-if analysis: recommended shift for high-risk flights (left) and the drop in predicted risk for moved flights (right).

falls from 0.488 to 0.441. The most common recommendation is to depart two hours earlier (3,651 flights), which reflects the build-up of delays during the day (Fig. 6). These are model-based estimates of association, not causal effects: moving thousands of flights would itself change congestion and delays.

X. Discussion and Limitations

The MLP learns the same signal as the Phase 1 models and gives the best-calibrated probabilities, but it does not beat XGBoost in ranking quality. Most of the predictive power comes from the engineered history features, not from the embeddings. Pre-departure prediction is limited by causes the data do not contain, such as weather, air traffic control and late inbound aircraft. Other limitations are the single year and city, the seasonal difference between validation and test months, and the simple, non-causal what-if analysis. We did not build a hybrid ML–DL model, and the optional temporal model was left out.

XI. Conclusion and Future Work

A compact MLP with entity embeddings, trained on leakage-free features with a date-based split, reaches a test PR AUC of 0.386, close to XGBoost (0.391), with the best-calibrated probabilities of all models and stable results across seeds. The ablation study shows that recent delay history drives the predictions and that embeddings add little beyond it. Natural next steps are forecast weather inputs, a sequence model over the ordered history of each aircraft or airport, and a hybrid that feeds learned embeddings to a tree model.

References

[1] A. Sternberg, J. Soares, D. Carvalho, and E. Ogasawara, "A review on flight delay prediction," arXiv preprint arXiv:1703.06118, 2017.

[2] Y. J. Kim, S. Choi, S. Briceno, and D. Mavris, "A deep learning approach to flight delay prediction," in Proceedings of the IEEE/AIAA 35th Digital Avionics Systems Conference (DASC), 2016.

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533–536, 1986.

[4] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in Proceedings of the 27th International Conference on Machine Learning, 2010, pp. 807–814.

[5] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in Advances in Neural Information Processing Systems 26, 2013, pp. 3111–3119.

[6] C. Guo and F. Berkhahn, "Entity embeddings of categorical variables," arXiv preprint arXiv:1604.06737, 2016.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, no. 56, pp. 1929–1958, 2014.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in International Conference on Learning Representations (ICLR), 2015.

[9] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in International Conference on Learning Representations (ICLR), 2019.

[10] L. Prechelt, "Early stopping — but when?" in Neural Networks: Tricks of the Trade. Springer, 1998, pp. 55–69.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[12] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track, 2022.

[13] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," Information Fusion, vol. 81, pp. 84–90, 2022.

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proceedings of the 34th International Conference on Machine Learning, 2017, pp. 1321–1330.

[15] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2002, pp. 694–699.

[16] H. Wickham, "nycflights13: Flights that departed NYC in 2013," https://github.com/tidyverse/nycflights13.

[17] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32, 2019, pp. 8024–8035.